

Guía Pedagógica: Día 10 - Infinite Scroll e Intersection Observer

Hoy implementaremos una técnica de optimización de carga fundamental en aplicaciones modernas: el **Infinite Scroll**. Aprenderemos a cargar contenido dinámicamente a medida que el usuario navega, evitando cargar cientos de elementos de golpe y mejorando drásticamente el rendimiento.

Contexto Pedagógico: Scroll vs Intersection Observer

Tradicionalmente, para detectar si el usuario llegó al final, escuchábamos el evento scroll.

- **Problema del evento scroll:** Se dispara cientos de veces por segundo. Escuchar este evento y calcular la posición del scroll constantemente puede hacer que la página se sienta lenta o pesada.
- **Solución: Intersection Observer:** Es una API moderna que permite "observar" un elemento específico. El navegador nos avisa solo en el momento exacto en que ese elemento entra (o sale) de la pantalla. Es extremadamente eficiente.

Paso 1: El Servicio - Soportar Paginación

La API de TMDb nos permite pedir los datos por páginas usando el parámetro `&page=X`. Actualizaremos nuestro servicio para permitirlo.

Abre **src/app/core/services/movie.service.ts**:

```
/**
 * Obtiene las películas populares con soporte para paginación.
 */
getPopularMovies(page: number = 1) {
  return this.http.get<MovieResponse>(`${this.apiUrl}/movie/popular`, {
    params: { page: page.toString() }
  });
}
```

Paso 2: La Lógica del Observador (El Cerebro)

Abre **src/app/features/home/home.ts**. Aquí es donde configuraremos al "vigía" que nos avisará cuando cargar más películas.

```
// 1. Necesitamos herramientas para interactuar con el DOM de forma segura
import { AfterViewInit, ElementRef, ViewChild, PLATFORM_ID } from '@angular/core';
import { isPlatformBrowser } from '@angular/common';

export class Home implements OnInit, AfterViewInit {
  // 2. Marcamos un elemento del HTML para observarlo
  @ViewChild('infiniteAnchor') infiniteAnchor!: ElementRef;

  catalogMovies = signal<Movie[]>([]);
  currentPage = signal(1);
  isFetchingNextPage = signal(false);

  // ... rest of logic ...

  ngAfterViewInit(): void {
    // 3. Solo configuramos el observador en el navegador (SSR Safety)
    if (isPlatformBrowser(this.platformId)) {
      this.initInfiniteScroll();
    }
  }

  private initInfiniteScroll(): void {
    const observer = new IntersectionObserver((entries) => {
      // 4. Si el ancla entra en el campo de visión y no estamos cargando...
      if (entries[0].isIntersecting && !this.isFetchingNextPage()) {
        this.loadMoreMovies();
      }
    }, { rootMargin: '200px' }); // 'rootMargin' permite cargar 200px antes de llegar al final

    observer.observe(this.infiniteAnchor.nativeElement);
  }

  loadMoreMovies(): void {
    this.isFetchingNextPage.set(true);
    this.movieService.getPopularMovies(this.currentPage()).subscribe({
      next: (data) => {
        // 5. Inmutabilidad: Concatenamos los resultados usando el operador spread [...]
        this.catalogMovies.set([...this.catalogMovies(), ...data.results]);
        this.currentPage.update(p => p + 1);
        this.isFetchingNextPage.set(false);
      }
    });
  }
}
```

Paso 3: El Ancla y el Catálogo (HTML)

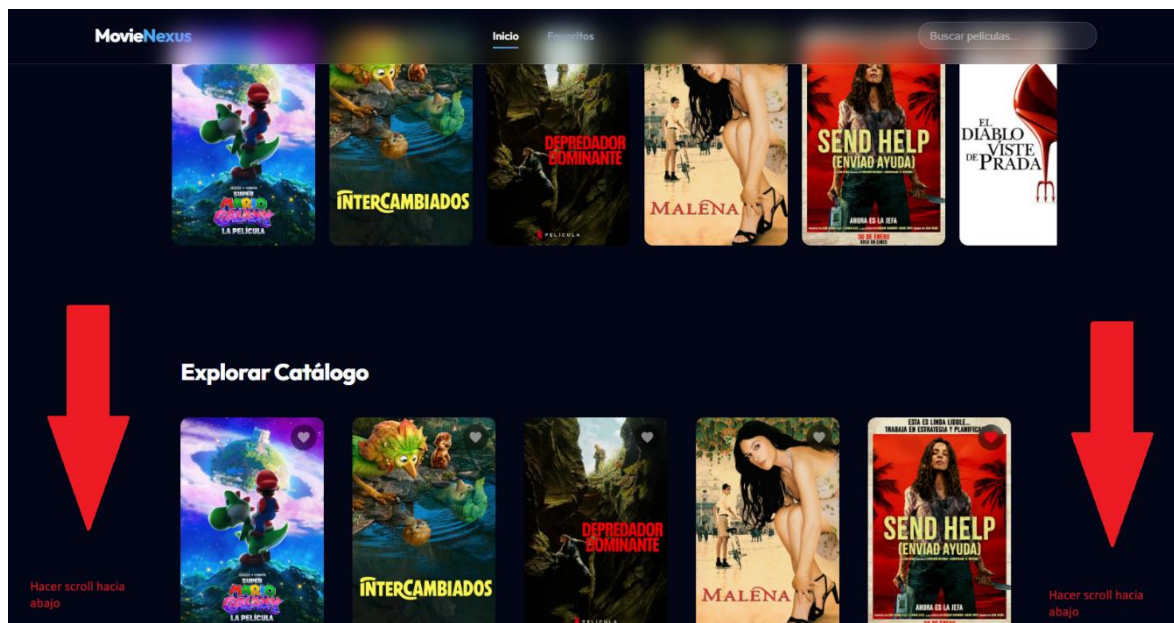
Abre `src/app/features/home/home.html`. Añadiremos el grid y el elemento que servirá de señal para el observador.

```
<section class="catalog-section">
  <h2 class="section-title">Explorar Catálogo</h2>

  <div class="movies-grid">
    @for (movie of catalogMovies(); track movie.id) {
      <app-movie-card [movie]="movie"></app-movie-card>
    }
  </div>

  <!-- El Ancla: Cuando este div entra en pantalla, se cargan más películas -->
  <div #infiniteAnchor class="infinite-anchor">
    @if (isFetchingNextPage()) {
      <div class="infinite-spinner"></div>
      <span>Cargando más películas...</span>
    }
  </div>
</section>
```

Verificamos la **Carga Infinita**: Ahora, al bajar por el Home, la aplicación carga automáticamente 20 películas más cada vez, creando un catálogo "sin fin"



Prueba de Aprendizaje (Checklist)

1. **¿Por qué el Intersection Observer es mejor que el evento scroll?** *(Respuesta: Porque no sobrecarga el hilo principal del navegador; solo se activa cuando hay una intersección real, ahorrando recursos de CPU).*
2. **¿Qué hace rootMargin: '200px' en la configuración del observador?** *(Respuesta: Crea un margen virtual. En este caso, dispara la carga de películas 200 píxeles antes de que el usuario llegue realmente al final, haciendo que la experiencia parezca instantánea).*
3. **¿Cómo concatenamos datos en una Signal sin perder los anteriores?** *(Respuesta: Usando el operador spread: signal.set([...anterior, ...nuevo])).*

Control de Versiones

Sube el avance

```
git add .
git commit -m "feat: implementar scroll infinito usando intersection observer en el home"
git push origin main
```